Guided Practice: React Integration

Outcome

You will create a Node and Express web application that uses React as a client-side scripting package to provide an interactive user interface to create a Tic Tac Toe game.

Resources Needed

- VCASTLE

-- OR --

- System configured for MEAN (technically MERN) stack development.

Level of Difficulty

Moderate

Deliverables

Deliverables are marked in red font or with a red picture border around the screenshot. **Your name should be visible as directed in screenshots that you submit.**

You will submit a link to your GitHub repository that contains files associated with this assignment. This will include:

1. Screenshot(s) of the application running (when screenshots are needed is called out in various steps in the document).
2. Committed code files.

Program Specifications

This practice program is designed to demonstrate and help you gain a deeper understanding of creating and running a web application that uses React for client-side scripting. This guided practice is based on the React.Dev tutorial found at https://react.dev/learn/tutorial-tic-tac-toe.

Development Steps

Confirm the Starter Code

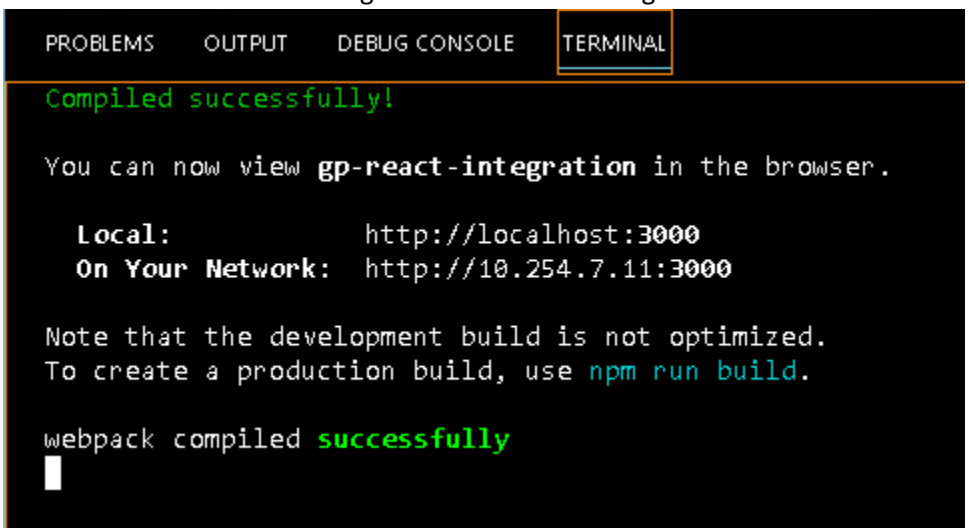
In this part of the guided practice, you will confirm the provided starter application code functions properly. This picks up where the guided practice instructions in Canvas leave off, which is with the project open in VS Code.

1. We're going to make sure the provided code works by running the app:
    a. Open a command (cmd) terminal in VS Code.
    b. Run the command:
    ```
    npm install
    ```

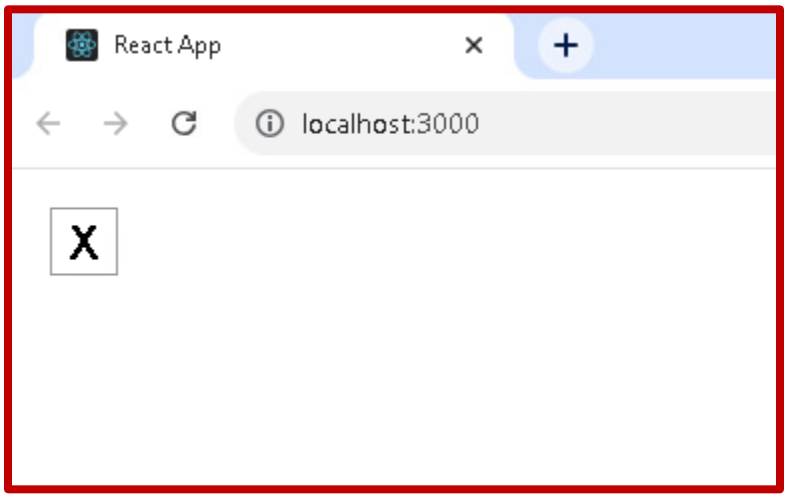
    You may see a series of warnings and a list of vulnerabilities - ignore these; do not address the issues.

    c. Once complete (it may take a few minutes to complete), start the server by running the command:
    ```
    npm start
    ```

This should result in something similar to the following:



d. A browser window pointed to http://localhost:3000/ should open automatically. If it does not, open a browser and navigate to that URL. The page should appear as follows:



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.

Understanding the Starter Code

Now that we've seen the stater code working (simple as it is…just displaying an "X"), let's examine what the starter code is actually doing. To understand the code, we'll look at 3 key files in the "src" folder:

- App.js
- styles.css
- index.js

App.js:

The code in App.js creates a *component*. In React, a component is a piece of reusable code that represents a part of a user interface. Components are used to render, manage, and update the UI elements in your application. Let's look at the component line by line to see what's going on:

```
export default function Square() {
  return <button className="square">X</button>;
}
```

The first line defines a function called Square. The export JavaScript keyword makes this function accessible outside of this file. The default keyword tells other files using your code that it's the main function in your file.

The second line returns a button. The return JavaScript keyword means whatever comes after is returned as a value to the caller of the function. <button> is a *JSX element*. A JSX element is a combination of JavaScript code and HTML tags that describes what you'd like to display. className="square" is a button property or *prop* that tells CSS how to style the button. X is the text displayed inside of the button and </button> closes the JSX element to indicate that any following content shouldn't be placed inside the button.

styles.css
This file defines the styles for your React app. The first two *CSS selectors* (* and body) define the style of large parts of your app while the .square selector defines the style of any component where the className property is set to square. In your code, that would match the button from your Square component in the App.js file.

index.js
You won't be editing this file during the guided practice, but it is the bridge between the component you created in the App.js file and the web browser.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './styles.css';

import App from './App';
```

Lines 1-5 bring all the necessary pieces together:

- React
- React's library to talk to web browsers (React DOM)
- the styles for your components
- the component you created in App.js.

The remainder of the file brings all the pieces together and injects the final product into index.html in the public folder.

Understanding the purpose and function of each file is important to the building of the remainder of the application. For the remainder of the guided practice, we'll stay in App.js and incrementally make updates that you'll see reflected live in the running application.

NOTE: if the application is not running and displayed in a web browser, start it before continuing with the next steps.

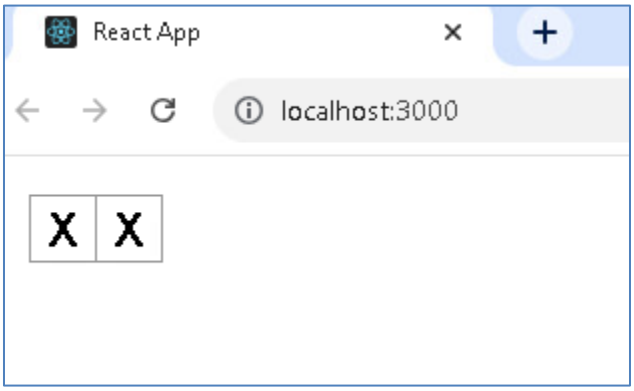
Building the Board

1. The first thing to do is add some more buttons to our board. We have one but we need 9 in total. React components need to return a single JSX element, so we can't just add multiple adjacent JSX elements to

get two buttons. We can use *fragments* (<> and </>) to wrap multiple adjacent JSX elements. Update the Square() function as follows:

```
export default function Square() {
  return (
    <>
      <button className="square">X</button>
      <button className="square">X</button>
    </>
  );
}
```

This should result in your web application displaying as follows - now we have 2 of our 9 buttons!



2. While it may be tempting to just copy/paste 7 more of the button lines and call it a day, that would result in something that's not so much like a tic tac toe board because it would appear as follows on the page:

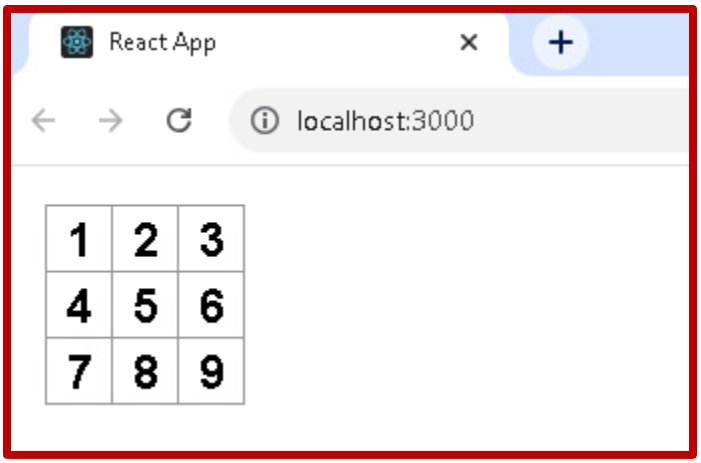
That's not quite right, so we won't do that. Instead we'll do a layout using tags and the className "board-row", which is defined in our .css file. We're also going to switch from X in each square to using numbered squares, just to make it easier to see which square is which on the board.

Since we'e no longer displaying a single square, let's also change the name of the function from "Square()" to "Board()". Update the App.js file to reflect the following code (this is the entirety of the App.js file at this point):

```
export default function Board() {
  return (
    <>
      <div className="board-row">
        <button className="square">1</button>
        <button className="square">2</button>
        <button className="square">3</button>
      </div>
      <div className="board-row">
        <button className="square">4</button>
        <button className="square">5</button>
        <button className="square">6</button>
      </div>
```

```
      <div className="board-row">
        <button className="square">7</button>
        <button className="square">8</button>
        <button className="square">9</button>
      </div>
    </>
  );
}
```

Save your changes and you'll see the application recompile, and your web page should now appear as follows:



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.

Passing Data Through Properties

1. We're now going to do a little re-architecting to take advantage of React's component-based architecture. Why? Well, when we go to an interactive model, if we wanted to change a value in a single square we'd need 9 copies of the code we currently have. That seems terribly inefficient and prone to errors (and it is). So, first, we're going to add our own component that represents a single square (sound familiar? it is).

   The differences between this and the original square component we had include:

   - This Square component is not exported
   - It is not default
   - It takes a function parameter called "value"

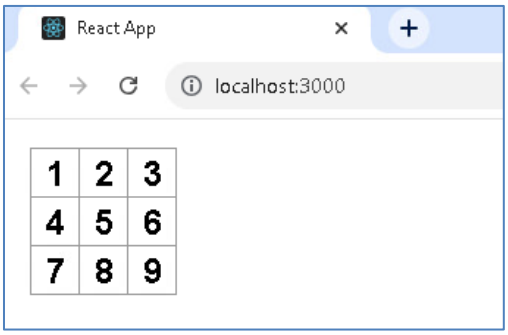
   Add this code above the Board() function in App.js:

```
function Square({ value }) {
  return <button className="square">{value}</button>
}
```

2. If you look at your web page now it won't look too good - just a single empty square. The reason for this is that we haven't updated the Board component to take advantage of our new Square component. Now we'll update Board() with the following code (note that we've replaced the button elements with Square components and given each a value):

```
export default function Board() {
  return (
    <>
      <div className="board-row">
        <Square value="1" />
        <Square value="2" />
        <Square value="3" />
      </div>
      <div className="board-row">
        <Square value="4" />
        <Square value="5" />
        <Square value="6" />
      </div>
      <div className="board-row">
        <Square value="7" />
        <Square value="8" />
        <Square value="9" />
      </div>
    </>
  );
}
```

Save your changes and you'll see the application recompile, and your web page should now appear just as it did before we added the component:



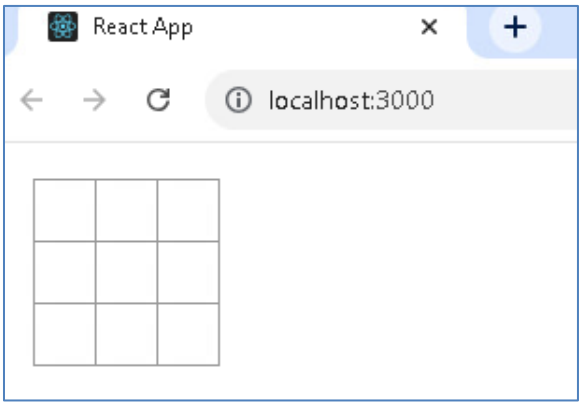
Making Components Interactive

1. So now we have a componentized React web application, but we still aren't playing Tic-Tac-Toe because we're just displaying a 3x3 grid with numbers. Now we're going to add some interactivity to our board. First, let's get rid of those values in the Board() function. Update the Board() function code as follows:

```
export default function Board() {
  return (
    <>
      <div className="board-row">
        <Square />
        <Square />
        <Square />
      </div>
      <div className="board-row">
        <Square />
```

```
        <Square />
        <Square />
      </div>
      <div className="board-row">
        <Square />
        <Square />
        <Square />
      </div>
    </>
  );
}
```

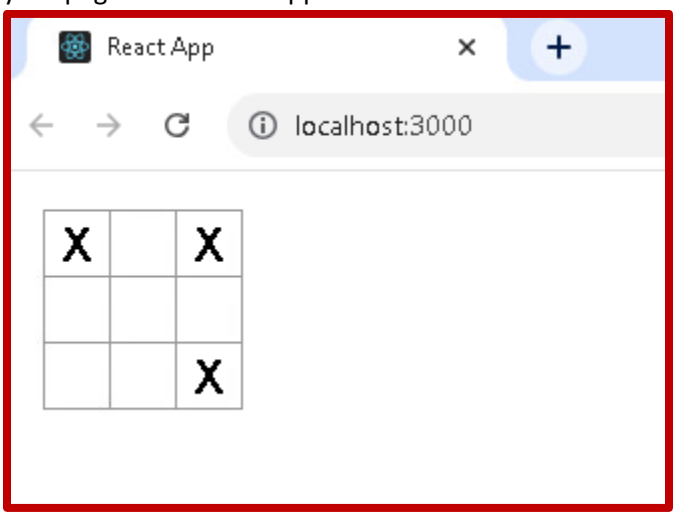
This should result in your web application displaying as follows - empty squares because we're no longer passing values to our Square() component.



2. Now we'll make things interactive by modifying the Square component to display an "X" when it's clicked. Note that we're adding a function "handleClick()" within our Square component. That function will simply set the value of the Square to X when the square is clicked. Update the Square component to reflect the following code (note also the addition of importing the "useState" function from the react library):

```
import { useState } from 'react';

function Square() {
  const [value, setValue] = useState(null);

  function handleClick() {
    setValue('X');
  }

  return (
    <button
      className="square"
      onClick={handleClick}
    >
      {value}
    </button>
  );
}
```

Save your changes and you'll see the application recompile. Click 3 random squares on the board, and your page should now appear as follows:



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.

Finishing the Game

Now we have an interactive web application with React, but just clicking around and having an X appear isn't really playing Tic-Tac-Toe and we don't handle anything related to the values that are already set - we just set whatever is clicked to "X". This section will cover the final updates to the App.js file, which include:

- Update the Board component to contain the function for handling the clicks and keep track of what's where on the board, allowing for interactive play.
- Update the Square component to accept an additional parameter to set as the onClick function.
- Add the ability to determine a winner of the game.

1. First we'll write a function to determine a winner. The basic processing is to create an array that contains all the winning possibilities then compare a passed-in array to the possibilities array and return the winning player or null.

   Create a function "calculateWinner" below the Board component and add the following code:

```
function calculateWinner(squares) {
  const lines = [
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8],
    [0, 3, 6],
    [1, 4, 7],
    [2, 5, 8],
    [0, 4, 8],
    [2, 4, 6],
  ];
  for (let i = 0; i < lines.length; i++) {
    const [a, b, c] = lines[i];
    if (squares[a] && squares[a] === squares[b] && squares[a] === squares[c]) {
      return squares[a];
```

```
    }
  }
  return null;
}
```

2. Next we'll update the Square component to take a click event handler rather than implementing it itself, allowing Board to keep track of everything that's going on on the board. Update the Square component as follows:

```
function Square({value, onSquareClick}) {
  return (
    <button className="square" onClick={onSquareClick}>
      {value}
    </button>
  );
}
```

3. Finally, we'll update Board to handle all of the Board state, turns, and checking for a winner. Update the Board component as follows:

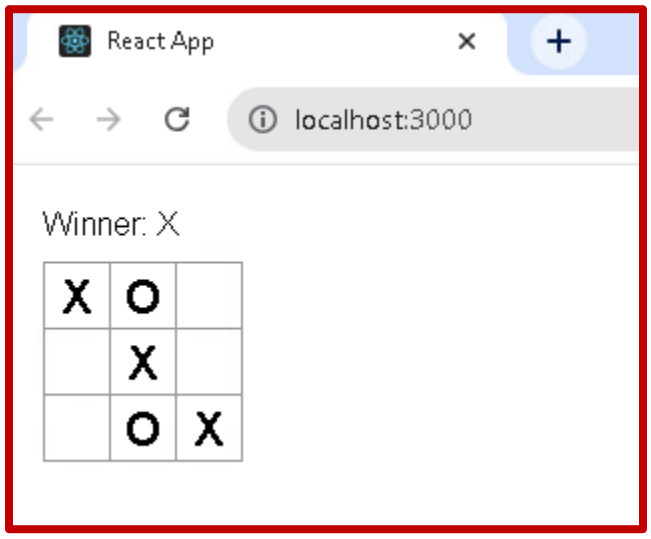
```
export default function Board() {
  const [xIsNext, setXIsNext] = useState(true);
  const [squares, setSquares] = useState(Array(9).fill(null));

  function handleClick(i) {
    if (calculateWinner(squares) || squares[i]) {
      return;
    }
    const nextSquares = squares.slice();
    if (xIsNext) {
      nextSquares[i] = 'X';
    } else {
      nextSquares[i] = 'O';
    }
    setSquares(nextSquares);
    setXIsNext(!xIsNext);
  }

  const winner = calculateWinner(squares);
  let status;
  if (winner) {
    status = 'Winner: ' + winner;
  } else {
    status = 'Next player: ' + (xIsNext ? 'X' : 'O');
  }

  return (
    <>
      <div className="status">{status}</div>
```

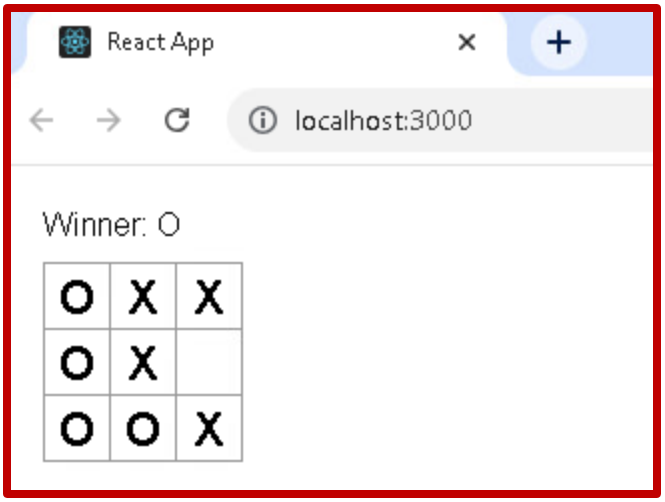
```
      <div className="board-row">
        <Square value={squares[0]} onSquareClick={() => handleClick(0)} />
        <Square value={squares[1]} onSquareClick={() => handleClick(1)} />
        <Square value={squares[2]} onSquareClick={() => handleClick(2)} />
      </div>
      <div className="board-row">
        <Square value={squares[3]} onSquareClick={() => handleClick(3)} />
        <Square value={squares[4]} onSquareClick={() => handleClick(4)} />
        <Square value={squares[5]} onSquareClick={() => handleClick(5)} />
      </div>
      <div className="board-row">
        <Square value={squares[6]} onSquareClick={() => handleClick(6)} />
        <Square value={squares[7]} onSquareClick={() => handleClick(7)} />
        <Square value={squares[8]} onSquareClick={() => handleClick(8)} />
      </div>
    </>
  );
}
```

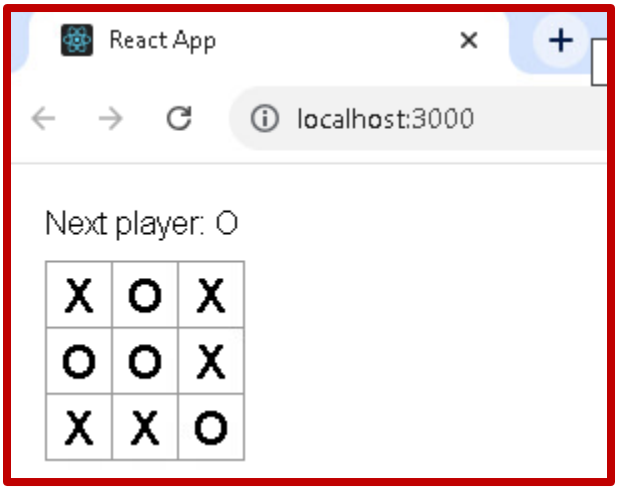
Save your changes and you'll see the application recompile.

Take 3 screenshots that resemble the ones below and save them in the same folder as your README.md file in the repository. They should show:

- X as the winner
- O as the winner
- No winner (draw)




React App
localhost:3000
Winner: O
O X X
O X
O O X


React App
localhost:3000
Next player: O
X O X
O O X
X X O